

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a ver
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/edwinwakanda/tugascitra/ea89bfee1efd875cca41982669b53591.jpg
/kaggle/input/datasets/edwinwakanda/tugascitra/WhatsApp Image 2026-06-19 at 23.46.28 (1).jpeg
```

```
import cv2 as cv
import matplotlib.pyplot as plt
import numpy as np

# Alamat path asli yang terdeteksi di Kaggle kamu
file1 = '/kaggle/input/datasets/edwinwakanda/tugascitra/ea89bfee1efd875cca41982669b53591.jpg'
file2 = '/kaggle/input/datasets/edwinwakanda/tugascitra/WhatsApp Image 2026-06-19 at 23.46.28 (1).jpeg'

# =====
# BAGIAN A: MEMBACA & KONVERSI CITRA
# =====

img1 = cv.imread(file1)
img2 = cv.imread(file2)

# Konversi BGR ke RGB
img1_rgb = cv.cvtColor(img1, cv.COLOR_BGR2RGB)
img2_rgb = cv.cvtColor(img2, cv.COLOR_BGR2RGB)

# Menampilkan Output Bagian A
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].imshow(img1)
axes[0, 0].set_title("Citra 1 Asli (BGR)")
axes[0, 0].axis('off')

axes[0, 1].imshow(img1_rgb)
axes[0, 1].set_title("Citra 1 Setelah Konversi (RGB)")
axes[0, 1].axis('off')

axes[1, 0].imshow(img2)
axes[1, 0].set_title("Citra 2 Asli (BGR)")
axes[1, 0].axis('off')

axes[1, 1].imshow(img2_rgb)
axes[1, 1].set_title("Citra 2 Setelah Konversi (RGB)")
axes[1, 1].axis('off')

plt.suptitle("Bagian A: Representasi Ruang Warna BGR vs RGB", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Metadata Informasi Citra
print("=== METADATA CITRA 1 ===")
print(f"Ukuran (Shape) : {img1_rgb.shape}")
print(f"Tipe Data : {img1_rgb.dtype}")
```

```

print(f"Nilai Min Pixel : {img1_rgb.min()}")
print(f"Nilai Max Pixel : {img1_rgb.max()}\n")

print("=== METADATA CITRA 2 ===")
print(f"Ukuran (Shape) : {img2_rgb.shape}")
print(f"Type Data : {img2_rgb.dtype}")
print(f"Nilai Min Pixel : {img2_rgb.min()}")
print(f"Nilai Max Pixel : {img2_rgb.max()}\n")

# =====
# BAGIAN B: RESIZE CITRA
# =====
target_size = (600, 400)
img1_resize = cv.resize(img1_rgb, target_size)
img2_resize = cv.resize(img2_rgb, target_size)

# Menampilkan Output Bagian B
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(img1_resize)
axes[0].set_title(f"Citra 1 Resized to {img1_resize.shape[:2]}")
axes[0].axis('off')

axes[1].imshow(img2_resize)
axes[1].set_title(f"Citra 2 Resized to {img2_resize.shape[:2]}")
axes[1].axis('off')

plt.suptitle("Bagian B: Hasil Penyesuaian Dimensi (Resize)", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# =====
# BAGIAN C: IMAGE BLENDING
# =====
# Eksperimen 3 kombinasi bobot
blend1 = cv.addWeighted(img1_resize, 0.2, img2_resize, 0.8, 0)
blend2 = cv.addWeighted(img1_resize, 0.5, img2_resize, 0.5, 0)
blend3 = cv.addWeighted(img1_resize, 0.8, img2_resize, 0.2, 0)

# Menampilkan Output Bagian C
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(blend1)
axes[0].set_title("Alpha=0.2, Beta=0.8\n(Citra 2 Dominan)")
axes[0].axis('off')

axes[1].imshow(blend2)
axes[1].set_title("Alpha=0.5, Beta=0.5\n(Seimbang)")
axes[1].axis('off')

axes[2].imshow(blend3)
axes[2].set_title("Alpha=0.8, Beta=0.2\n(Citra 1 Dominan)")
axes[2].axis('off')

plt.suptitle("Bagian C: Eksperimen Variasi Bobot Image Blending", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

```

-----
error                                Traceback (most recent call last)
/tmp/ipykernel_1562/1450430880.py in <cell line: 0>()
    14
    15 # Konversi BGR ke RGB
--> 16 img1_rgb = cv.cvtColor(img1, cv.COLOR_BGR2RGB)
    17 img2_rgb = cv.cvtColor(img2, cv.COLOR_BGR2RGB)
    18

error: OpenCV(4.13.0) /io/opencv/modules/imgproc/src/color.cpp:199: error: (-215:Assertion failed) !_src.empty() in
function 'cvtColor'

```

Next steps: [Explain error](#)

Pertanyaan Analisis Bagian A:

1. Mengapa warna citra dapat terlihat tidak sesuai jika langsung ditampilkan setelah dibaca dengan OpenCV?

- **Jawaban:** Karena secara bawaan (default), OpenCV membaca dan menyimpan citra dalam urutan kanal warna BGR (Blue, Green, Red). Sedangkan library visualisasi Matplotlib membaca urutan kanal warna dalam format RGB (Red, Green, Blue). Jika kita langsung menampilkan citra BGR menggunakan Matplotlib tanpa konversi, warna merah dan biru akan saling tertukar sehingga visualisasinya terlihat aneh.

2. Apa perbedaan BGR dan RGB?

- **Jawaban:** Perbedaannya terletak pada urutan penyimpanan array matriks warnanya. Pada format RGB, indeks 0 mewakili kanal Merah (Red), indeks 1 mewakili Hijau (Green), dan indeks 2 mewakili Biru (Blue). Sebaliknya, pada format BGR urutannya dibalik: indeks 0 adalah Biru, indeks 1 adalah Hijau, dan indeks 2 adalah Merah.

3. Mengapa penting memahami format warna sebelum melakukan manipulasi citra?

- **Jawaban:** Memahami format warna sangat krusial karena setiap operasi manipulasi piksel didasarkan pada perhitungan matematis per kanal warna. Jika kita salah mengenali urutan format warna, manipulasi yang kita lakukan (seperti pengubahan kontras, segmentasi warna, atau penggabungan gambar) akan menghasilkan kalkulasi nilai piksel yang salah dan merusak kualitas visual objek asli.

```
# =====
# BAGIAN D: IMAGE NEGATIVE & HISTOGRAM
# =====

# 1. Membuat citra negatif menggunakan rumus matematika (255 - I)
img1_negative = 255 - img1_resize

# 2. Menampilkan Citra Asli vs Citra Negatif beserta Histogramnya
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# [Baris 1, Kolom 1] Tampilkan Citra Asli (RGB yang sudah di-resize)
axes[0, 0].imshow(img1_resize)
axes[0, 0].set_title("Citra Asli (RGB)")
axes[0, 0].axis('off')

# [Baris 1, Kolom 2] Tampilkan Citra Negatif
axes[0, 1].imshow(img1_negative)
axes[0, 1].set_title("Citra Negatif")
axes[0, 1].axis('off')

# [Baris 2, Kolom 1] Tampilkan Histogram Citra Asli per Kanal Warna
colors = ('r', 'g', 'b')
for i, col in enumerate(colors):
    hist = cv.calcHist([img1_resize], [i], None, [256], [0, 256])
    axes[1, 0].plot(hist, color=col, linewidth=2)
axes[1, 0].set_title("Histogram Citra Asli")
axes[1, 0].set_xlim([0, 256])
axes[1, 0].set_xlabel("Nilai Intensitas Pixel")
axes[1, 0].set_ylabel("Jumlah Pixel")
axes[1, 0].grid(True, linestyle='--', alpha=0.6)

# [Baris 2, Kolom 2] Tampilkan Histogram Citra Negatif per Kanal Warna
for i, col in enumerate(colors):
    hist_neg = cv.calcHist([img1_negative], [i], None, [256], [0, 256])
    axes[1, 1].plot(hist_neg, color=col, linewidth=2)
axes[1, 1].set_title("Histogram Citra Negatif")
axes[1, 1].set_xlim([0, 256])
axes[1, 1].set_xlabel("Nilai Intensitas Pixel")
axes[1, 1].set_ylabel("Jumlah Pixel")
axes[1, 1].grid(True, linestyle='--', alpha=0.6)

plt.suptitle("Bagian D: Implementasi Operasi Image Negative dan Analisis Histogram", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Bagian D: Implementasi Operasi Image Negative dan Analisis Histogram

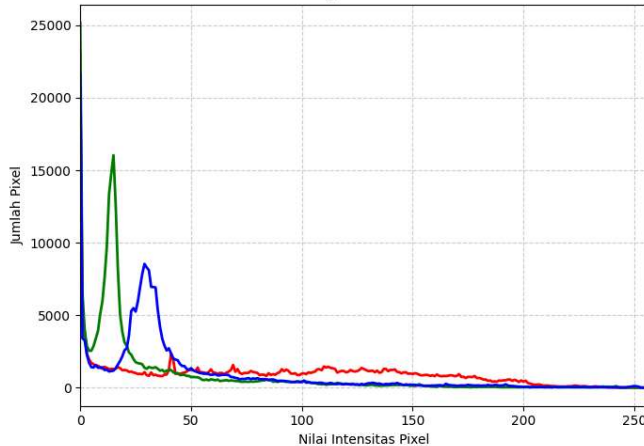
Citra Asli (RGB)



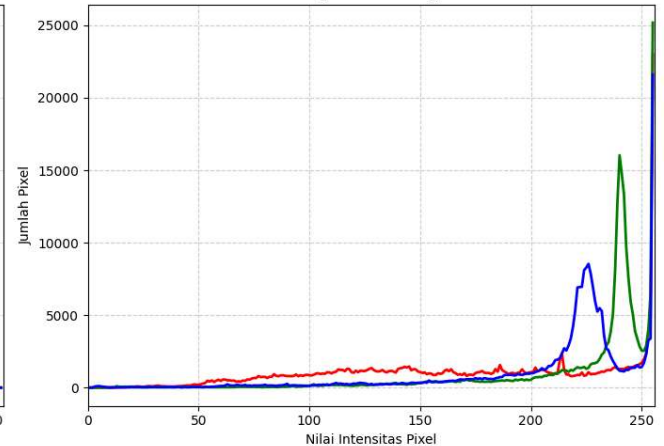
Citra Negatif



Histogram Citra Asli



Histogram Citra Negatif



▼ **Pertanyaan Analisis Bagian D:**

1. Apa yang terjadi pada pixel terang setelah dilakukan operasi negatif?

- **Jawaban:** Pixel terang yang memiliki nilai intensitas tinggi (mendekati 255) akan berbalik menjadi pixel gelap (mendekati 0) karena hasil dari operasi pengurangan matematika $255 - \text{nilai asli}$.

2. Apa yang terjadi pada pixel gelap setelah dilakukan operasi negatif?

- **Jawaban:** Pixel gelap yang memiliki nilai intensitas rendah (mendekati 0) akan berbalik menjadi pixel yang sangat terang (mendekati 255).

3. Bagaimana perubahan histogram setelah citra dibuat negatif?

- **Jawaban:** Grafik histogram akan mengalami pencerminan secara horizontal (efek *flipping*). Pola sebaran pixel yang tadinya menumpuk di sisi kiri (area gelap) akan berpindah secara simetris ke sisi kanan (area terang), begitu pula sebaliknya.

4. Dalam kondisi apa image negative dapat berguna?

- **Jawaban:** Sangat berguna pada pengolahan citra medis (seperti memperjelas detail tulang atau jaringan pada foto rontgen/X-Ray) serta untuk menonjolkan detail fitur atau teks tersembunyi pada area gambar yang tertutup bayangan hitam pekat.

```
# =====
# BAGIAN E: TRANSFORMASI LOGARITMIK
# =====

# 1. Menghitung konstanta c berdasarkan nilai pixel maksimum citra asli
# Rumus: c = 255 / log10(1 + r_max)
r_max = np.max(img1_resize)
c_log = 255.0 / np.log10(1.0 + r_max)
```

```

# 2. Menerapkan rumus Transformasi Logaritmik:  $s = c * \log_{10}(1 + r)$ 
img1_log = c_log * np.log10(1.0 + img1_resize)

# 3. Mengubah tipe data kembali menjadi uint8 (0-255)
img1_log = np.array(img1_log, dtype=np.uint8)

# 4. Menampilkan Hasil Visualisasi Citra dan Histogram Perbandingan
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# [Baris 1, Kolom 1] Citra Asli
axes[0, 0].imshow(img1_resize)
axes[0, 0].set_title("Citra Asli (RGB)")
axes[0, 0].axis('off')

# [Baris 1, Kolom 2] Citra Hasil Transformasi Logaritmik
axes[0, 1].imshow(img1_log)
axes[0, 1].set_title("Citra Hasil Transformasi Logaritmik")
axes[0, 1].axis('off')

# [Baris 2, Kolom 1] Histogram Citra Asli
colors = ('r', 'g', 'b')
for i, col in enumerate(colors):
    hist = cv.calcHist([img1_resize], [i], None, [256], [0, 256])
    axes[1, 0].plot(hist, color=col, linewidth=2)
axes[1, 0].set_title("Histogram Citra Asli")
axes[1, 0].set_xlim([0, 256])
axes[1, 0].set_xlabel("Nilai Intensitas Pixel")
axes[1, 0].set_ylabel("Jumlah Pixel")
axes[1, 0].grid(True, linestyle='--', alpha=0.6)

# [Baris 2, Kolom 2] Histogram Citra Logaritmik
for i, col in enumerate(colors):
    hist_log = cv.calcHist([img1_log], [i], None, [256], [0, 256])
    axes[1, 1].plot(hist_log, color=col, linewidth=2)
axes[1, 1].set_title("Histogram Citra Logaritmik")
axes[1, 1].set_xlim([0, 256])
axes[1, 1].set_xlabel("Nilai Intensitas Pixel")
axes[1, 1].set_ylabel("Jumlah Pixel")
axes[1, 1].grid(True, linestyle='--', alpha=0.6)

plt.suptitle("Bagian E: Implementasi Transformasi Logaritmik dan Analisis Histogram", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 5. Menampilkan Perbandingan Nilai Pixel Minimum dan Maksimum
print("=== PERBANDINGAN NILAI PIXEL (SABALUM VS SESUDAH) ===")
print(f"Citra Asli      -> Min Pixel: {img1_resize.min()}, Max Pixel: {img1_resize.max()}")
print(f"Citra Logarithm  -> Min Pixel: {img1_log.min()}, Max Pixel: {img1_log.max()}")
print(f"Nilai Konstanta Skala (c): {c_log:.4f}")

```

Bagian E: Implementasi Transformasi Logaritmik dan Analisis Histogram

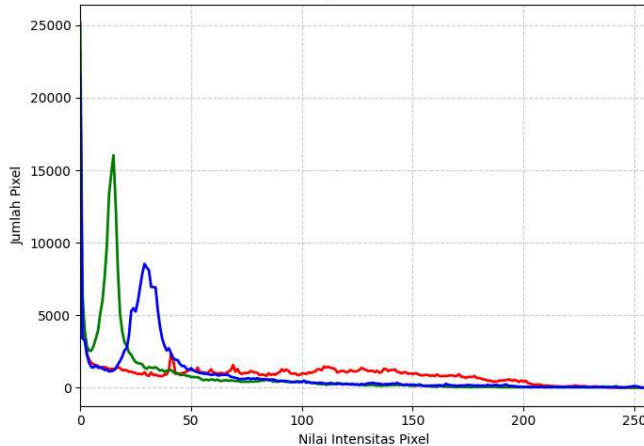
Citra Asli (RGB)



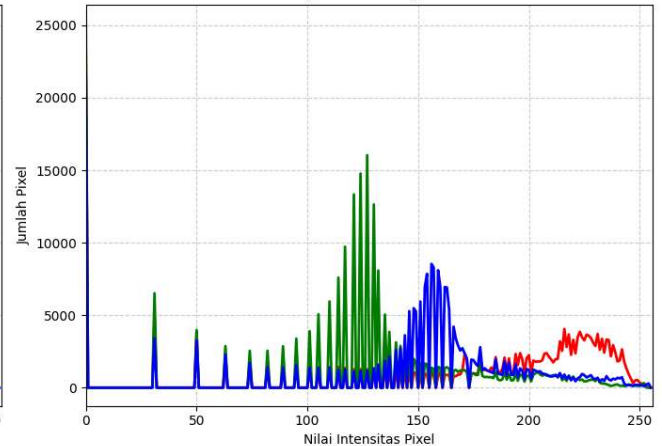
Citra Hasil Transformasi Logaritmik



Histogram Citra Asli



Histogram Citra Logaritmik



=== PERBANDINGAN NILAI PIXEL (SABALUM VS SESUDAH) ===
 Citra Asli -> Min Pixel: 0, Max Pixel: 255
 Citra Logarithm -> Min Pixel: 0, Max Pixel: 255
 Nilai Konstanta Skala (c): 105.8865

▼ Pertanyaan Analisis Bagian E:

1. Apa efek transformasi logaritmik terhadap area gelap pada citra?

- **Jawaban:** Transformasi logaritmik akan memperluas (*stretch*) rentang nilai pixel yang rendah (area gelap). Efek visualnya, area yang tadinya gelap atau remang-remang akan dipetakan ke tingkat kecerahan yang lebih tinggi, sehingga detail objek di area gelap menjadi jauh lebih jelas terlihat.

2. Apa efek transformasi logaritmik terhadap area terang pada citra?

- **Jawaban:** Terhadap area terang, transformasi logaritmik akan melakukan kompresi (*compress*) rentang nilai pixel tinggi. Artinya, area yang sudah terang tidak akan bertambah terang secara signifikan, melainkan diredam agar tidak terlalu mengalami overexposure.

3. Mengapa transformasi logaritmik sering digunakan untuk meningkatkan detail pada area gelap?

- **Jawaban:** Karena karakteristik fungsi logaritma memiliki kurva yang naik sangat tajam pada nilai input (r) rendah, kemudian melandai pada nilai input tinggi. Sifat matematis ini sangat ideal untuk mendistribusikan ulang variasi nilai piksel gelap secara lebih merata ke rentang grayscale yang lebih luas.

4. Apa risiko transformasi logaritmik jika digunakan pada citra yang sudah sangat terang?

- **Jawaban:** Jika digunakan pada citra yang sudah dominan terang, risikonya adalah penurunan kontras secara drastis (*washed out*). Citra akan kehilangan detail pada area putih karena gradasi warna pudar dan gambar cenderung terlihat keputihan/berkabut secara keseluruhan.

```
# =====
# BAGIAN F: TRANSFORMASI GAMMA
```

```
# =====

# 1. Normalisasi citra asli ke rentang 0.0 - 1.0 (wajib untuk komputasi Gamma)
img_normalized = img1_resize / 255.0

# 2. Menentukan 4 variasi nilai gamma sesuai instruksi tugas
gamma_values = [0.5, 1.0, 2.0, 2.5]

# 3. Menyiapkan plot visualisasi (2 baris x 4 kolom: Baris 1 untuk Citra, Baris 2 untuk Histogram)
fig, axes = plt.subplots(2, 4, figsize=(20, 10))
colors = ('r', 'g', 'b')

for idx, g in enumerate(gamma_values):
    # Rumus Transformasi Gamma: s = c * (r ^ gamma), dengan c = 1
    gamma_corrected = np.power(img_normalized, g)

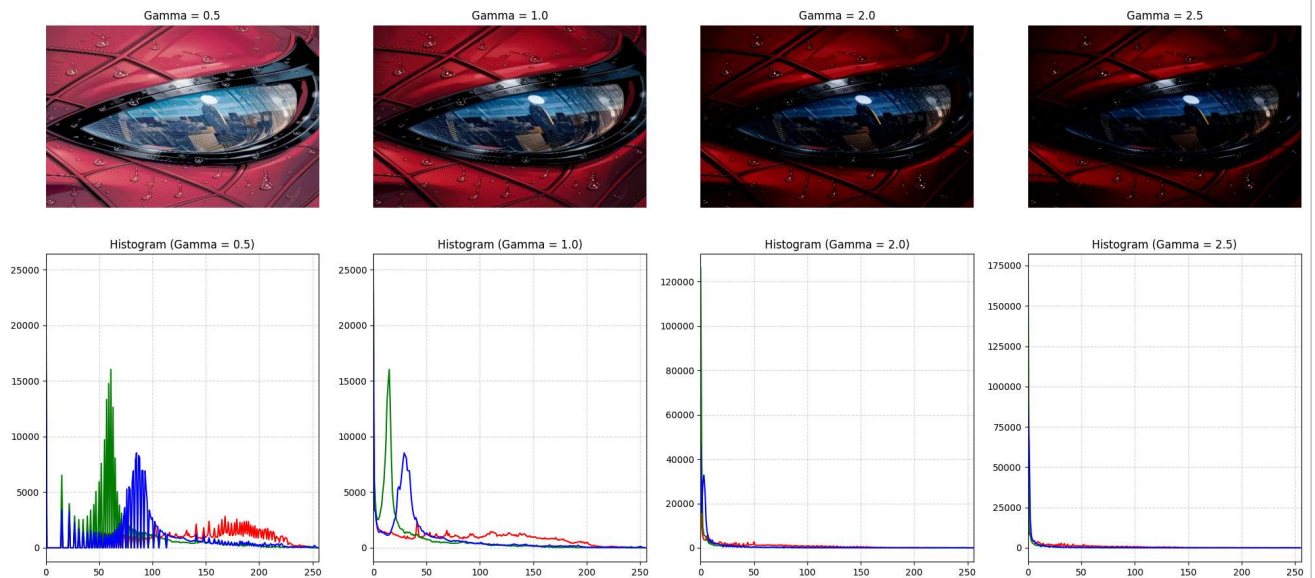
    # Kembalikan skala ke rentang 0-255 dan ubah tipe data ke uint8
    gamma_corrected = np.array(gamma_corrected * 255.0, dtype=np.uint8)

    # [Baris 1] Plot Citra Hasil Gamma
    axes[0, idx].imshow(gamma_corrected)
    axes[0, idx].set_title(f"Gamma = {g}")
    axes[0, idx].axis('off')

    # [Baris 2] Plot Histogram per Kanal Warna
    for i, col in enumerate(colors):
        hist_gamma = cv.calcHist([gamma_corrected], [i], None, [256], [0, 256])
        axes[1, idx].plot(hist_gamma, color=col, linewidth=1.5)
    axes[1, idx].set_title(f"Histogram (Gamma = {g})")
    axes[1, idx].set_xlim([0, 256])
    axes[1, idx].grid(True, linestyle='--', alpha=0.5)

plt.suptitle("Bagian F: Eksperimen Koreksi Gamma Non-Linear dan Pergeseran Histogram", fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

Bagian F: Eksperimen Koreksi Gamma Non-Linear dan Pergeseran Histogram



Pertanyaan Analisis Bagian F:

1. Apa yang terjadi pada citra ketika nilai $\gamma < 1$?

- **Jawaban:** Citra akan menjadi lebih terang secara keseluruhan. Skala pemetaan non-linear akan menarik kurva ke atas sehingga pixel-pixel gelap dinaikkan intensitasnya secara signifikan tanpa merusak area putih.

2. Apa yang terjadi pada citra ketika nilai $\gamma > 1$?

- **Jawaban:** Citra akan menjadi lebih gelap. Kurva pemetaan melandai di awal, menyebabkan pixel berintensitas rendah dan menengah ditekan ke bawah menjadi lebih gelap.
3. Bagaimana bentuk histogram bergeser untuk setiap nilai gamma yang diuji?
- **Jawaban:** Pada $\gamma = 0.5$, grafik histogram bergeser secara non-linear ke arah kanan (area terang). Pada $\gamma = 1.0$, histogram tetap/identik dengan aslinya. Sedangkan pada $\gamma = 2.0$ dan 2.5 , grafik histogram menumpuk dan bergeser ke arah kiri (area gelap).
4. Apa perbedaan utama efek visual antara transformasi logaritmik dan transformasi gamma?
- **Jawaban:** Transformasi logaritmik memiliki satu kurva tetap yang sangat agresif dalam menaikkan area gelap namun rentan membuat citra terlihat pudar (*washed out*). Sementara transformasi gamma jauh lebih fleksibel karena tingkat kecerahan atau kegelapan dapat diatur secara presisi melalui variabel eksponen γ .
5. Kapan sebaiknya kita menggunakan transformasi gamma dibanding teknik lainnya?
- **Jawaban:** Ketika kita ingin melakukan kalibrasi kecerahan layar monitor (*display correction*), memperbaiki foto yang terlalu silau (*overexposed*), atau mencerahkan foto *low-light* secara natural tanpa kehilangan detail kontras pada area ekstrem.

✓ BAGIAN G: ANALISIS HOTS & STUDI KASUS

G.1 Perbandingan Teknik Manipulasi Citra

Berikut adalah ringkasan karakteristik perbandingan dari keempat teknik yang telah dieksperimentkan:

Teknik Manipulasi	Jenis Operasi (Linear/Non-linear)	Efek Visual Utama pada Citra	Kapan Sebaiknya Digunakan?
Image Blending	Linear	Penggabungan transparan dua citra berdasarkan bobot intensitas.	Efek transparan, watermarking, at
Image Negative	Linear	Pembalikan warna secara simetris (gelap jadi terang, terang jadi gelap).	Citra medis (X-Ray) atau memperj
Transformasi Logaritmik	Non-linear	Kenaikan kecerahan sangat agresif pada area gelap, kompresi area terang.	Citra dengan rentang dinamis san
Transformasi Gamma	Non-linear	Koreksi kecerahan fleksibel dan melengkung halus sesuai nilai eksponen γ .	Kalibrasi perangkat layar, perbaiki

G.2 Analisis Pengambilan Keputusan (HOTS)

1. Kondisi Berawan & Minim Cahaya (Underexposed):

- **Rekomendasi:** Transformasi Gamma ($\gamma = 0.5$).
- **Alasan:** Berbeda dengan Logaritmik yang terlalu agresif sehingga bisa membuat detail awan yang sudah agak terang menjadi putih kabur (*washed out*), Transformasi Gamma dengan $\gamma < 1$ mampu mencerahkan area gelap secara gradual dan natural tanpa merusak kontras area menengah (*midtone*s).

2. Detail pada Area Sangat Terang (Overexposed/Silau):

- **Rekomendasi:** Transformasi Gamma ($\gamma = 2.0$ atau 2.5).
- **Alasan:** Nilai $\gamma > 1$ akan menekan nilai pixel tinggi ke arah bawah (gelap) secara non-linear. Hal ini secara efektif meredam kilau cahaya ekstrem (*glare*) dan memunculkan kembali detail tekstur objek yang tadinya tertutup oleh warna putih silau.

3. Kelemahan Utama Transformasi Logaritmik:

- **Analisis:** Kelemahan utamanya adalah kehilangan kontras secara drastis pada area citra yang sudah terang. Karena sifat kurva logaritma yang mengompresi nilai pixel tinggi secara ekstrem, citra yang dominan terang akan terlihat keputihan, pudar, dan kehilangan detail tekstur (*washed out*).

4. Pemberian Watermark Semi-Transparan:

- **Analisis:** Kita harus menggunakan **Image Blending** (`cv.addWeighted`) dengan memberikan bobot α yang besar pada citra utama (misal 0.9) dan bobot β yang sangat kecil pada citra logo/watermark (misal 0.1). Cara ini memastikan logo terlihat samar sebagai penanda hak cipta tanpa mengganggu estetika visual citra utama.

5. Kaitan Pergeseran Histogram dengan Kualitas Visual:

- **Analisis:** Histogram adalah representasi statistik sebaran pixel. Jika grafik histogram menumpuk di kiri, citra tampak gelap (*low-contrast/underexposed*). Jika menumpuk di kanan, citra tampak terlalu silau (*overexposed*). Operasi manipulasi citra

yang baik idealnya bertujuan menyebarkan grafik histogram secara merata ke seluruh rentang (0-255) agar citra memiliki tingkat kecerahan (*brightness*) dan kontras yang optimal di mata manusia.

G.3 Analisis Studi Kasus (Pilihan Kasus 1 & 3)

- **Studi Kasus 1: Perbaikan Citra Satelit Malam Hari (Low-Light)**
 - *Solusi Teknis:* Menggunakan **Transformasi Logaritmik** atau **Transformasi Gamma** ($\gamma = 0.4$).
 - *Argumentasi:* Citra satelit malam hari memiliki karakteristik rentang dinamis yang sangat timpang (banyak area hitam pekat, namun ada titik lampu kota yang sangat terang). Transformasi logaritmik atau gamma dengan nilai kecil sangat efektif untuk memperluas (*stretch*) nilai pixel gelap secara drastis agar pola geografis daratan terlihat, sementara titik lampu kota diredam agar tidak menjadi bercak putih rusak.
 - **Studi Kasus 3: Inspeksi Cacat Keretakan Logam Industri**
 - *Solusi Teknis:* Menggunakan **Image Negative** ($255 - I$).
 - *Argumentasi:* Pada komponen logam, retakan mikro sering kali tersembunyi di dalam bayangan atau celah-celah gelap berintensitas rendah. Dengan menerapkan operasi Image Negative, area retakan yang gelap akan dibalik menjadi putih terang yang kontras di atas latar belakang logam yang kini berubah menjadi keabu-abuan. Hal ini mempermudah operator industri maupun algoritma komputer visi untuk mendeteksi letak cacat produksi secara presisi.
-

BAGIAN H: REFLEKSI PRIBADI

1. **Konsep Paling Menarik:** Konsep transformasi non-linear (Logaritmik dan Gamma). Sangat menakjubkan melihat bagaimana rumus matematika sederhana bisa mengubah pencahayaan sebuah foto secara cerdas tanpa merusak komponen warna aslinya.
2. **Kendala Terbesar:** Memahami visualisasi histogram dan menanganikan error pembacaan path file input di Kaggle. Sava